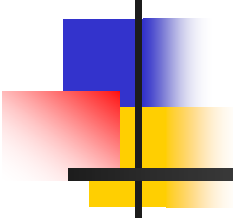
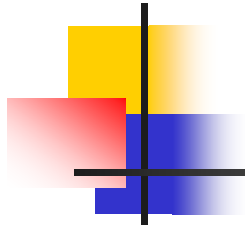


DATA STRUCTURES AND ALGORITHMS



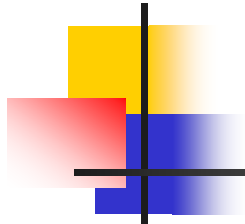
Lecture Notes 8

Pradit Pitaksathienkul



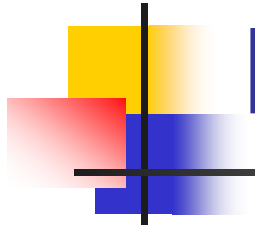
ROAD MAP

- **PRIORITY QUEUES (HEAPS)**
 - **Model**
 - **Simple Implementations**
 - **Binary Heap**

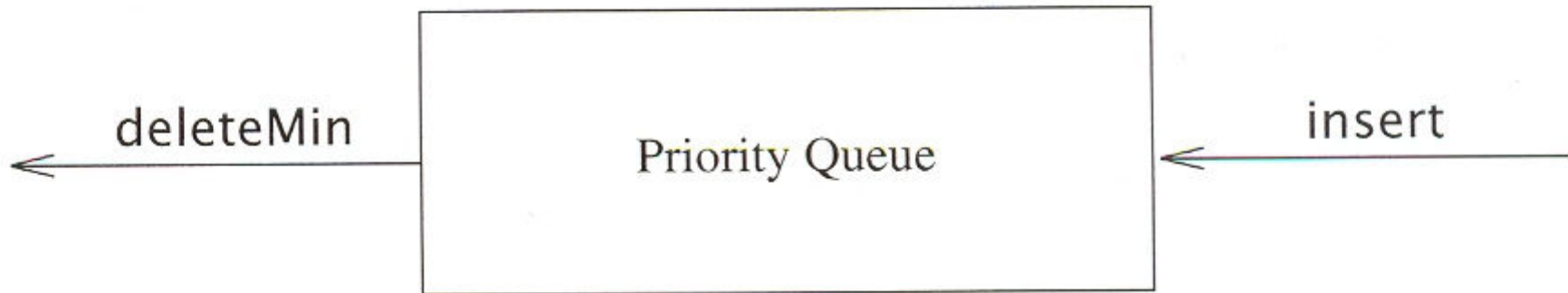


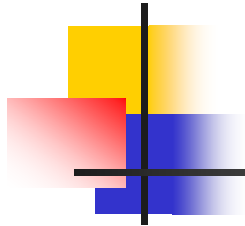
Model

- Priority Queue is a data structure that allows at least two operations
 - *insert*
 - equivalent to *enqueue* operation in queue ADT
 - *deleteMin*
 - finds, returns and removes the minimum element in priority queue
 - equivalent to *dequeue* operation in queue ADT



Model

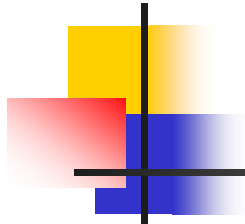




Model

Many applications

- Printer queues
- Multiuser O/S process scheduling
- Many graph algorithms



Heap Structures

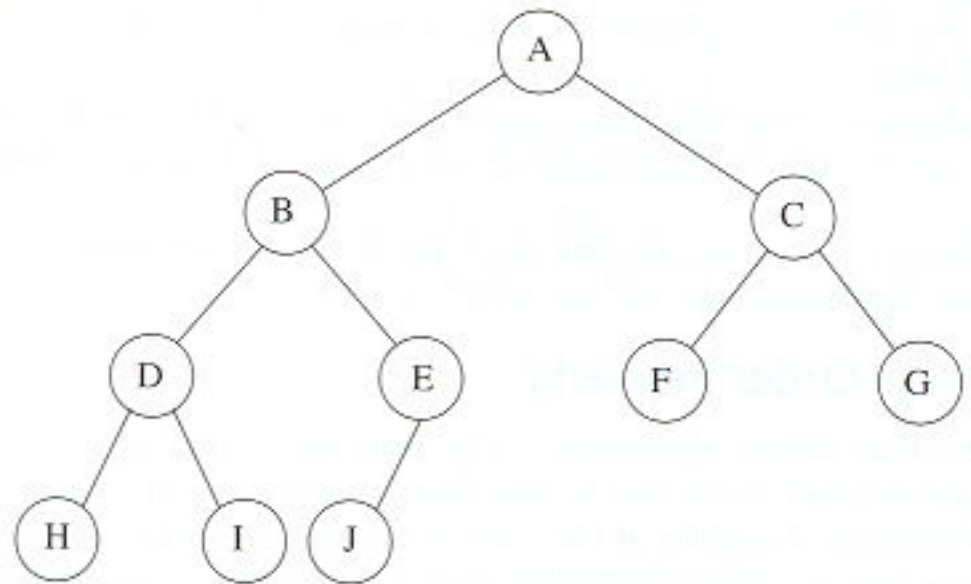
- **Binary Heap** : we learn this one.
- d-Heap
- Leftist Heap
- Binomial Heap
- Fibonacci Heap

Binary heap is very common !

- We will refer to binary heaps as “heaps”
- Heaps have 2 properties คุณสมบัติ
 - Structure property คุณสมบัติเชิงโครงสร้าง
 - Heap-Order property คุณสมบัติเชิงอันดับ

Structure Property of Heaps

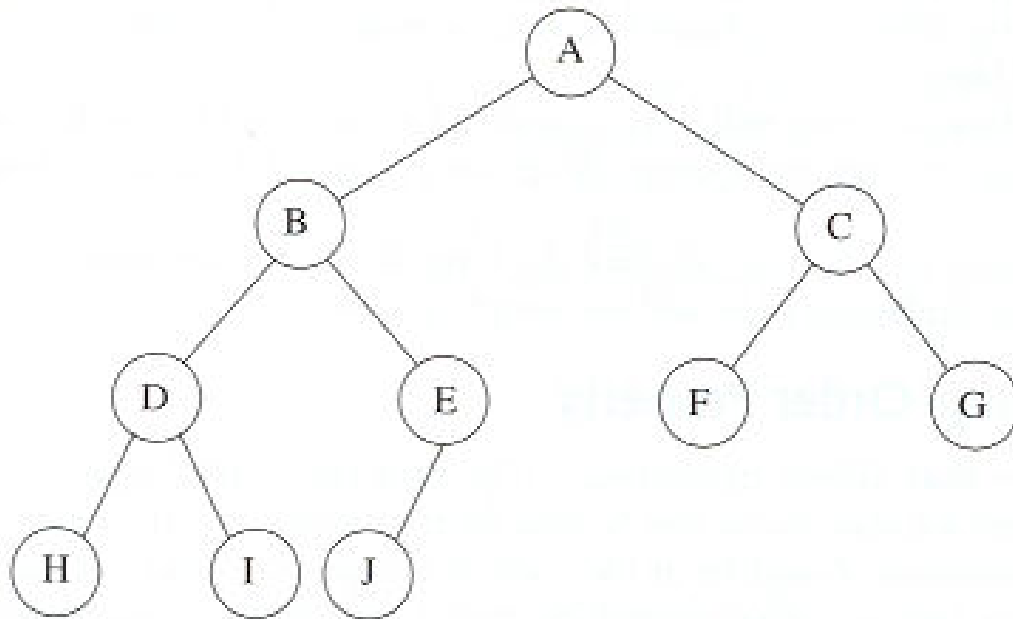
- Binary heap is a complete binary tree
 - completely filled except the bottom level which is filled from left to right
- if heap has **height h** then **number of nodes** is between 2^h and $2^{h+1} - 1$
- height of a heap with N nodes is $O(\log N)$



complete binary tree
it is a regular structure so can be
represented in an array !

Structure Property of Heaps

Array implementation of complete binary tree



- For an element in position i
 - $2i$ is the left child
 - $2i+1$ is the right child
 - $i/2$ is the parent

ตัวอย่างที่ 7/2 = 3

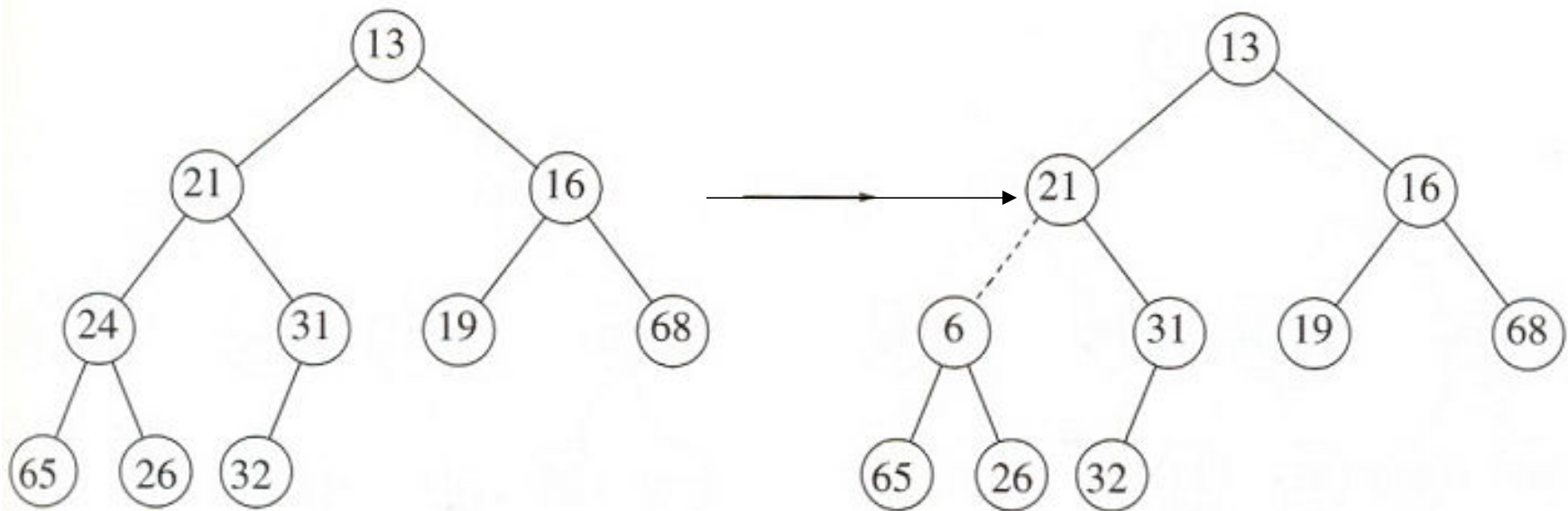
Python use //

Map tree to array
implementation

	A	B	C	D	E	F	G	H	I	J			
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Heap-Order Property

- For each node X , key in the **parent is smaller** than the key in X (except the root)
- Minimum element can always be found at the root



Only left tree is a heap !

```
class BinaryHeap:
def __init__(self, capacity=100):
# สร้าง list ที่มีขนาด capacity + 1 (index 0 ไม่ถูกใช้งาน)
    self.array = [None] * (capacity + 1)
    self.currentSize = 0

def is_empty(self):
    return self.currentSize == 0

def is_full(self):
    return self.currentSize == len(self.array) - 1

def find_min(self):
    if self.is_empty():
        print("Queue is empty")
        return None
    return self.array[1]
```